den; fragen Sie Ihren Administrator (denjenigen, der das zentrale Archiv angelegt hat), was Sie hier genau eintragen müssen.

H.4 An einem Projekt gemeinsam arbeiten

Nachdem ein Benutzer ein gemeinsames Projekt im Archiv angelegt hat, können andere Teammitglieder darauf zugreifen. Dazu müssen sie nur im Menü WERKZEUGE/TEAMARBEIT den Menüpunkt ARBEITSKOPIE ERSTELLEN auswählen, der dafür sorgt, dass eine Kopie des gemeinsamen Projekts vom zentralen Server auf ihrem Rechner kopiert wird. Danach können sie auf der lokalen Kopie arbeiten.

H.5 Aktualisieren und Abgeben

Ab und zu müssen die verschiedenen Kopien des Projekts, an denen die Teammitglieder auf ihren eigenen Rechnern arbeiten, synchronisiert werden. Dies geschieht über das zentrale Archiv. Mit der Funktion ÄNDERUNGEN ABGEBEN kopieren Sie ihre Änderungen in das Archiv und mit der Funktion AKTUALSIERUNG ABHOLEN kopieren Sie die Änderungen aus dem Archiv (die andere Teammitglieder vorgenommen haben) in Ihre lokale Kopie. Es empfiehlt sich, das Projekt möglichst oft zu aktualisieren und abzugeben, sodass die Änderungen nicht zu umfangreich werden.

H.6 Weitere Informationen

Weitere Informationen finden Sie in dem Teamarbeit-Tutorial auf das Sie über BLUEJ-TUTORIAL im HILFE-Menü von BlueJ zugreifen können.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 11:05 Uhr

ANHANG I Javadoc

Gutes Dokumentieren von Klassen und Interfaces ist eine wichtige Ergänzung zum Schreiben hochwertiger Quelltexte. Mit der Dokumentation können Sie menschlichen Lesern Ihre Vorstellungen auf einem hohen sprachlichen Niveau vermitteln, anstatt sie zum Lesen des relativ primitiven Quelltextes zu zwingen. Dokumentation ist insbesondere für die öffentlichen Anteile von Klassen und Interfaces nützlich, da ein Programmierer diese idealerweise benutzen kann, ohne die Details der Implementierung kennen zu müssen.

In allen Beispielprojekten dieses Buches haben wir einen speziellen Kommentierungsstil verwendet, der von dem Dokumentationswerkzeug *Javadoc* erkannt wird. Javadoc wird zusammen mit dem JDK ausgeliefert. Dieses Werkzeug automatisiert die konsistente Generierung von Klassendokumentationen in Form von HTML-Seiten. Das API von Java wurde mit demselben Werkzeug generiert und man weiß dies zu schätzen, wenn man die Bibliotheksklassen benutzt.

In diesem Anhang geben wir einen kurzen Überblick über die wichtigsten Dokumentationskommentare, die Sie in die Routine Ihrer eigenen Softwareentwicklung integrieren sollten.

I.1 Dokumentationskommentare

Die zu dokumentierenden Teile einer Klasse sind die Klassendefinition als Ganzes sowie ihre Datenfelder, Konstruktoren und Methoden. Für den Benutzer einer Klasse ist die Dokumentation der Klasse sowie der öffentlichen Konstruktoren und Methoden am wichtigsten. Wir neigen dazu, Datenfelder nicht im Javadoc-Stil zu kommentieren, weil wir sie als Implementierungsdetails ansehen, auf die sich ein Benutzer nicht verlassen sollte.

Dokumentationskommentare werden immer mit der Zeichenfolge `/**` eingeleitet und mit `*/` abgeschlossen. Zwischen diesen Symbolen kann ein Kommentar aus einer *Übersichtsbeschreibung*, gefolgt von einem *Markierungsabschnitt* bestehen, die beide optional sind.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 11:05 Uhr

I.1.1 Die Übersichtsbeschreibung

Die Übersichtsbeschreibung einer Klasse sollte eine generelle Beschreibung des Klassenzwecks sein. Listing I.1 zeigt den Teil einer typischen Übersichtsbeschreibung aus der Klasse `Spiel` des *Zuul*-Projektes. Beachten Sie, dass die Beschreibung Details darüber enthält, wie diese Klasse zum Starten des Spiels zu benutzen ist.

Listing I.1
Die Übersichtsbeschreibung eines Klassenkommentars.

```
/**
 *  Dies ist die Hauptklasse der Anwendung "Die Welt von Zuul".
 *  "Die Welt von Zuul" ist ein sehr einfaches, textbasiertes
 *  Adventure-Game. Ein Spieler kann sich in einer Umgebung bewegen,
 *  mehr nicht. Das Spiel sollte auf jeden Fall ausgebaut werden,
 *  damit es interessanter wird!
 *
 *  Zum Spielen muss eine Instanz dieser Klasse erzeugt werden und
 *  an ihr die Methode "spielen" aufgerufen werden.
 */
```

Die Übersichtsbeschreibung für eine Methode sollte recht allgemein gehalten sein, ohne auf Details ihrer Implementierung einzugehen. Tatsächlich benötigt die Übersichtsbeschreibung einer Methode häufig nur einen einzelnen Satz, wie etwa

```
/**
 * Erzeuge einen neuen Fahrgast mit verschiedener Abhol und
 * Zielposition.
 */
```

Besonders sorgfältig sollte der erste Satz (bis zum ersten Punkt) der Übersichtsbeschreibung einer Klasse, eines Interface oder einer Methode formuliert sein, da dieser in einem eigenen Übersichtsabschnitt zu Beginn der generierten Dokumentation eingetragen wird.

Javadoc unterstützt auch die Benutzung von HTML-Markierungen in Kommentaren.

I.1.2 Der Markierungsabschnitt

Im Anschluss an die Übersichtsbeschreibung folgt der Markierungsabschnitt. Javadoc erkennt rund 20 Markierungen, von denen wir hier nur die wichtigsten ansprechen wollen (Tabelle I.1). Markierungen können in zwei Formen auftreten: als *Blockmarkierung* und als *Zeilenmarkierung*. Wir werden lediglich Blockmarkierungen betrachten, weil diese am häufigsten benutzt werden. Weitere Details über Zeilenmarkierungen und zusätzliche Blockmarkierungen können dem Abschnitt über Javadoc der Dokumentation *Tools and Utilities* entnommen werden, die Teil des JDK ist.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 11:05 Uhr

Markierung	Verknüpfter Text
`@author`	Autorenname(n)
`@param`	Parametername und -beschreibung
`@return`	Beschreibung des Ergebnisses
`@see`	Querverweise
`@throws`	Geworfener Exception-Typ mit Kontext
`@version`	Versionsbeschreibung

Tabelle I.1
Häufig genutzte Javadoc-Markierungen.

Die Markierungen **`@author`** und **`@version`** sind üblicherweise in den Kommentaren von Klassen und Interfaces zu finden, für Konstruktoren, Methoden und Datenfelder sind sie nicht vorgesehen. Beide werden gefolgt von Freitext, an den keine speziellen Anforderungen gestellt werden. Beispiele sind

```
@author Hacker T. Largebrain
@version 2004.12.31
```

Die Markierungen **`@param`** und **`@throws`** werden in Methoden und Konstruktoren verwendet, während **`@return`** ausschließlich in Methoden zulässig ist. Beispiele sind

```
@param limit der maximal zulässige Wert.
@return eine Zufallszahl aus dem Bereich 1 bis limit (inklusiv).
@throws IllegalLimitException wenn limit kleiner als 1 ist.
```

Die Markierung **`@see`** kennt verschiedene Formate und kann in jedem Dokumentationskommentar verwendet werden. Es bietet die Möglichkeit, Querverweise von einem Kommentar zu einer anderen Klasse oder Methode oder zu einer anderen Form der Dokumentation zu formulieren. Ein solcher Querverweis führt dazu, dass in der Dokumentation das dokumentierte Element um einen Abschnitt *See Also* ergänzt wird. Hier sind einige typische Beispiele:

```
@see "Die Java Sprachspezifikation, von Joy et al."
@see <a href=http://www.bluej.org/>The BlueJ web site</a>
@see #istLebendig
@see java.util.ArrayList#add
```

Das erste Beispiel bettet einfach nur einen Text ohne Hyperlink ein; das zweite bettet einen Hyperlink zum genannten Dokument ein; das dritte verbindet mit der Dokumentation der Methode **`istLebendig`** innerhalb derselben Klasse; das vierte setzt einen Querverweis auf die Methode **`add`** in der Klasse **`java.util.ArrayList`**.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 11:05 Uhr

I.2 Unterstützung für Javadoc in BlueJ

Wenn ein Projekt im Javadoc-Stil kommentiert wurde, dann bietet BlueJ Unterstützung bei der Generierung der kompletten HTML-Dokumentation. Selektieren Sie im Hauptfenster den Menüeintrag WERKZEUGE/DOKUMENTATION ERZEUGEN und die Dokumentation wird generiert und in einem Browserfenster angezeigt.

Im BlueJ-Editor kann von der Quelltextsicht auf die Dokumentationssicht umgeschaltet werden, indem die Option *Implementierung* im Fenster oben rechts auf *Schnittstelle* umgestellt wird (Abbildung I.1). Dies ermöglicht eine schnelle Voransicht der Dokumentation, die jedoch keine Referenzen auf Superklassen oder benutzte Klassen enthält.

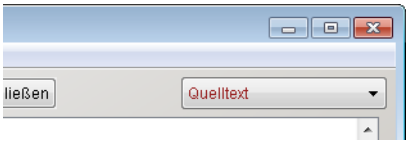

Abbildung I.1
Die Implementierung/Schnittstelle-Option im BlueJ-Editor.

Weitergehende Informationen sind verfügbar unter:

http://www.oracle.com/technetwork/java/javase/documentation/index-137868.html

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 11:05 Uhr

ANHANG

J

Quelltextkonventionen

J.1 Benennung

J.1.1 Sprechende Namen verwenden

Benutzen Sie sprechende Namen für alle Bezeichner (Klassen-, Variablen- und Methodennamen). Vermeiden Sie Mehrdeutigkeiten. Vermeiden Sie Abkürzungen. Einfache verändernde Methoden sollten in der Form *setzeIrgendetwas(…)* benannt werden. Einfache sondierende Methoden sollten in der Form *gibIrgendetwas(…)* benannt werden. Sondierende Methoden mit einem booleschen Ergebnis sind oft mit *istIrgendetwas(…)* benannt, beispielsweise `istLeer()`.

J.1.2 Klassennamen beginnen mit einem Großbuchstaben

J.1.3 Klassennamen sind Hauptwörter im Singular

J.1.4 Methoden und Variablennamen beginnen mit einem Kleinbuchstaben

Benutzen Sie für alle drei – für Klassen-, Methoden- und Variablennamen – Großbuchstabe am Anfang eines neuen Wortes nach dem ersten Wort, um zusammengesetzte Namen lesbarer zu machen, etwa `anzahlEintraege`.

J.1.5 Konstanten werden in GROSSBUCHSTABEN geschrieben

Verwenden Sie Unterstriche, um zusammengesetzte Konstantennamen lesbarer zu gestalten:

`OBERE_GRENZE`[1]

1 Anmerkung des Übersetzers: Im Deutschen ist dies aufgrund zusammengesetzter Hauptwörter nicht so häufig notwendig wie im Englischen.

Lizenziert für Hochschule Luzern, 147.88.200.39 am 22.02.2023 um 11:05 Uhr